

1. Since the adversary is not restricted to output equal-length messages we assume a situation when the adversary outputs

① $m_0 \in \{0, 1\}^*$, which is a one bit message with the length of the ciphertext at most polynomial $q(n)$

② $m_1 \in \{0, 1\}^{q(n)+2}$, which is a uniform message with the length of $q(n)+2$

When the challenger encrypts m_0 , the length of the ciphertext at most $q(n)$, which means $|c| = \text{Enc}_K(m_0) \leq q(n)$

When the challenger encrypts m_1 , let's assume the length of the ciphertext is also at most $q(n)$. In this case, the number of possible ciphertexts is:

$$2^0 + 2^1 + 2^2 + \dots + 2^{q(n)} = 1 \cdot \frac{1-2^{q(n)+1}}{1-2} = 2^{q(n)+1} - 1$$

However, the number of possible plaintext is:

It means that the ciphertext space is smaller than the plaintext space, which makes two plaintext can encrypt into the same ciphertext, and it makes decryption impossible.

In other words, there exists at least one plaintext whose ciphertext length is greater than $q(n)$.

Therefore, the adversary can guess which message being encrypted by looking at the length of the ciphertext

If $|c| \leq q(n)$, m_0 is being encrypted

If $|c| > q(n)$, m_1 is being encrypted.

This breaks random guess of $\Pr[\text{PrivKey}_{\pi}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$

2. The new scheme:

Key generation: Same as the old scheme where $K \in \{0, 1\}^n$

Encryption: Do padding to the plaintext until the length of the plaintext is $l(n)$. In this case, we assume the length of plaintext before padding is at most $l(n)-1$.

Padding rule: append zero with the length of one and ones until the length is $l(n)$. Then do the old scheme encryption

Decryption: Perform old scheme decryption and remove padding.
The last zero is the plaintext indicate the padding starts from here.

3. (a) G' is not necessarily a PRG.

Counterexample: $G(x) = \begin{cases} 0^n & \text{if } x \text{ starts with } 0^n \\ \text{pseudorandom} & \text{(else)} \end{cases}$

G is still PRG because it's very rare that input starts with 0^n .

G' is not PRG because output always starts with 0^n .

(b) G' is not necessarily a PRG

Counterexample: $G(s) = \text{pseudorandom} \parallel \text{last bit of } s$

$$G'(s) = G(s) \parallel G(s+1)$$

$$= \text{pseudorandom} \parallel s \parallel \text{pseudorandom} \parallel s'$$

Therefore, when the last bit in the first half output is always different than the last bit in the second half output, we can tell it comes from G' .

4. If G is not a PRG, then there exists an efficient distinguisher D can tell the difference between an truly random outputs and output produced by G . In this case, we can make truly random as G output XOR t , where t is a random string. Then, there at least one G outputs XOR t must behave differently from G output.

Therefore, we can choose two message $m_0 = \{0\}^n$, $m_1 = t$.

By Construction 3.17:

$$\text{Enc}_K(m_0) = G(K) \oplus m_0 = G(K) \oplus \{0\}^n = G(K)$$

$$\text{Enc}_K(m_1) = G(K) \oplus m_1 = G(K) \oplus t$$

Because D can tell the difference between $G(K)$ and $G(K) \oplus t$, D can distinguish which message was encrypted.

Therefore, G does not have indistinguishable encryptions.

5. Let $\ln(n) = \log n$. Then the number of possible inputs of the length $\log n$ is $2^{\log n} = n$. Since there are only n possible inputs, we do not need to construct some algorithm like PRG or PRF under assumptions. Instead, we can create a table where the inputs are all possible inputs from 0 to n . Each output is a random coin flip with either 0 or 1. After that, we can create a key with n bits where each bit stores the output for one input. Therefore, we implement a uniform function $F: \{0,1\}^n \times \{0,1\}^{\log n} \rightarrow \{0,1\}$ which is a PRF without any conditions.

6. No, F' is not a PRF. We can define a distinguisher D that choose $z \in \{0,1\}^{n-2}$, and two inputs $x_1 = z1$, $x_2 = 0z$

$$F'_k(x_1) = F_k(0 \| x_1) \| F_k(x_1 \| 1) \\ = F_k(0 \| z1) \| F_k(z1 \| 1)$$

$$F'_k(x_2) = F_k(0 \| x_2) \| F_k(x_2 \| 1) \\ = F_k(0 \| 0z) \| F_k(0z \| 1)$$

Because $0 \| z1 = 0z \| 1$, it means the first half of $F'_k(x_1)$ is always equal to the second half of $F'_k(x_2)$.

$$\text{So } \Pr[D^{F'}] = 1 = 1$$

When F' is a PRF, the probability of the first half output string equal to the second half of the output string is $2^{-n} = 2^{-n}$. Therefore the distinguisher has advantage $1 - 2^{-n}$, which is not negligible. Then F' is not a PRF.

7. Assume G is not a PRG, then there is some distinguishers D that can tell the difference between $G(\ln)$ and random string. Umm. We use D to build a distinguisher A against the PRF F . Given oracle access to a function O , the distinguisher A queries $O(\langle 1 \rangle)$, $O(\langle 2 \rangle)$, ..., $O(\langle m \rangle)$, concatenates the answers into $y = O(\langle 1 \rangle) \| O(\langle 2 \rangle) \| \dots \| O(\langle m \rangle)$, and runs D on y . If $O = F_s$ for a random seed s , then $y = F_s(\langle 1 \rangle) \| F_s(\langle 2 \rangle) \| \dots \| F_s(\langle m \rangle) = G(s)$, so D is seeing an output of G . If O is a random function,

then because $\langle 1 \rangle, \langle 2 \rangle, \dots, \langle m \rangle$ are distinct inputs, the outputs $O(\langle 1 \rangle), O(\langle 2 \rangle), \dots, O(\langle m \rangle)$ are random n -bit strings, so y is uniform in $\{0, 1\}^{mn}$. Therefore A distinguishes F from a truly random function with the same non-negligible probability as D . It contradicts the assumption that F is a PRF. So our assumption that G is not a PRG is wrong. G is a PRG.